

UNIVERSIDADE DE MOGI DAS CRUZES
BACHARELADO EM SISTEMAS DE INFORMAÇÃO

GUILHERME YOSHIZAWA DOS SANTOS

GUSTAVO YOSHIZAWA DOS SANTOS

ReUseHub

Mogi das Cruzes

2026

GUILHERME YOSHIZAWA DOS SANTOS

GUSTAVO YOSHIZAWA DOS SANTOS

ReUseHub

Trabalho desenvolvido para finalização do curso de Bacharelado em Sistemas de Informação da Universidade de Mogi das Cruzes, sob orientação do Prof. Bruno Messias Aguiar e coorientação do Prof. Alessandro Aparecido da Silva, contendo software e documentação de Projeto de Final de Curso para a obtenção de título de Bacharel em Sistemas de Informação.

Mogi das Cruzes

2026

RESUMO

O presente trabalho descreve o desenvolvimento do ReUseHub, um sistema web responsivo voltado à intermediação de doações e trocas de itens usados. O sistema contempla o cadastro e autenticação de usuários, publicação e gerenciamento de anúncios, busca com filtros por categoria e localização, mecanismo de destaque por relevância e atividade, chat entre usuários e sistema de avaliações e reputação. A solução adota arquitetura em camadas baseada no padrão MVC, com frontend desacoplado desenvolvido em React com TypeScript e Tailwind CSS, e backend em Java 21 com Spring Boot, comunicando-se por meio de API RESTful com autenticação via JWT. A persistência de dados é realizada de forma híbrida: PostgreSQL para dados estruturados, como usuários, anúncios e avaliações, e MongoDB para dados não estruturados, como mensagens de chat, logs e históricos de interações. O sistema adota controle de acesso baseado em perfis de usuário, com criptografia de dados sensíveis e aplicação dos princípios da Lei Geral de Proteção de Dados (LGPD). Integra ainda as APIs externas ViaCEP e OpenStreetMap Nominatim para validação e preenchimento automático de endereços, apoiando os filtros de busca por localização. O projeto inclui testes automatizados com JUnit e Mockito e disponibiliza dashboards analíticos ao administrador da plataforma.

Palavras-chave: marketplace de doações; economia circular; trocas de itens; Spring Boot; React.js; PostgreSQL; MongoDB; JWT; LGPD.

ABSTRACT

This work describes the development of ReUseHub, a responsive web system designed to facilitate the intermediation of donations and exchanges of used items. The system covers user registration and authentication, advertisement publication and management, search with filters by category and location, relevance-based highlight mechanisms, user-to-user chat, and a ratings and reputation system. The solution adopts a layered architecture based on the MVC pattern, with a decoupled frontend built with React.js, TypeScript, and Tailwind CSS, and a Java 21 Spring Boot backend communicating through a RESTful API with JWT authentication. Data persistence follows a hybrid approach: PostgreSQL for structured data such as users, advertisements, and reviews, and MongoDB for unstructured data such as chat messages, logs, and interaction histories. The system implements role-based access control with encryption of sensitive data and compliance with Brazil's General Data Protection Law (LGPD). It also integrates the ViaCEP and OpenStreetMap Nominatim external APIs for address validation and auto-fill, supporting location-based search filters. The project includes automated tests using JUnit and Mockito and provides analytical dashboards for platform administrators.

Keywords: donation marketplace; circular economy; item exchange; Spring Boot; React.js; PostgreSQL; MongoDB; JWT; LGPD.

1. INTRODUÇÃO

O descarte inadequado de objetos ainda em condições de uso representa um problema ambiental e social de crescente relevância. Milhões de itens utilizáveis são descartados anualmente no Brasil, contribuindo para o acúmulo de resíduos sólidos e o desperdício de recursos naturais. Nesse contexto, plataformas digitais que incentivem a reutilização de bens por meio de doações e trocas surgem como instrumentos importantes para a promoção da economia circular e do consumo consciente (ELLEN MACARTHUR FOUNDATION, 2023).

A popularização dos dispositivos móveis e o crescimento do acesso à internet no Brasil ampliaram significativamente o potencial de adoção de soluções digitais voltadas à intermediação de transações entre pessoas. Plataformas de marketplace, mesmo aquelas sem fins comerciais diretos, dependem de mecanismos robustos de busca, filtragem, comunicação e confiança entre os usuários para garantir sua adoção e sustentabilidade (OECD, 2019).

O presente trabalho propõe o desenvolvimento do ReUseHub, um sistema web responsivo para intermediação de doações e trocas de itens usados. A plataforma permite o cadastro de usuários, a publicação e gerenciamento de anúncios, a busca com filtros por categoria e localização, a comunicação direta entre usuários por meio de chat e a construção de reputação por meio de avaliações mútuas. O sistema adota arquitetura em camadas com separação entre frontend e backend, banco de dados híbrido e conformidade com a Lei Geral de Proteção de Dados (LGPD).

1.1. Objetivos

1.1.1 Objetivo Geral

Desenvolver um sistema web responsivo para intermediação de doações e trocas de itens usados, com mecanismos de busca, comunicação entre usuários e avaliação de reputação, visando promover a economia circular e o reaproveitamento de bens.

1.1.2 Objetivos Específicos

- Implementar o cadastro, autenticação e gerenciamento de usuários com controle de acesso baseado em perfis.
- Desenvolver o módulo de publicação e gerenciamento de anúncios de doação e troca de itens.
- Criar sistema de busca com filtros por categoria e localização geográfica.

- Implementar mecanismo de destaque de anúncios com base em relevância e atividade recente.
- Desenvolver módulo de chat para comunicação direta entre usuários interessados.
- Implementar sistema de avaliações e reputação de usuários.
- Garantir conformidade com a LGPD no tratamento de dados pessoais e sensíveis.
- Integrar APIs externas para validação e preenchimento automático de endereços.
- Disponibilizar dashboard analítico para o perfil administrador da plataforma.

2. REVISÃO DA LITERATURA

2.1 Economia Circular e Plataformas de Reuso

A economia circular é um modelo econômico que propõe a eliminação de resíduos e a circulação contínua de materiais e produtos, em contraposição ao modelo linear de produção e descarte. A Ellen MacArthur Foundation (2023) destaca que a transição para modelos circulares pode gerar benefícios ambientais expressivos, reduzindo a extração de recursos naturais e as emissões de gases de efeito estufa associadas à produção de novos bens.

No âmbito digital, plataformas de marketplace para doação e troca de itens usados constituem instrumentos práticos de implementação dos princípios circulares. Ao reduzir a barreira de acesso entre quem deseja descartar um objeto e quem pode utilizá-lo, essas plataformas criam valor social e ambiental sem necessariamente envolver transações financeiras (ELLEN MACARTHUR FOUNDATION, 2023).

2.2 Sistemas de Informação e Plataformas Digitais

Segundo a Britannica (2026), um sistema de informação é um conjunto integrado de componentes destinado a coletar, armazenar e processar dados, além de fornecer informações, conhecimentos e produtos digitais. No contexto de plataformas de intermediação, esses sistemas são o alicerce que permite a gestão coordenada de hardware, software e dados para apoiar as operações de negócio e a distribuição de informações entre usuários e anúncios.

Conforme a OECD (2019), plataformas digitais são modelos que criam valor ao facilitar interações entre dois ou mais grupos de usuários. Em plataformas de reuso,

a confiança entre os participantes é um fator crítico, sendo os sistemas de avaliação e reputação mecanismos essenciais para sua construção e manutenção.

2.3 Arquitetura de Software para Sistemas Web Modernos

A arquitetura de Monolito Modular representa uma abordagem intermediária entre o monolito tradicional e os microsserviços. Enquanto o monolito convencional tende a centralizar toda a lógica sem separação clara de domínios, e os microsserviços distribuem a aplicação em processos independentes com alta complexidade operacional, o Monolito Modular organiza o sistema em módulos internamente coesos e fracamente acoplados, mantendo um único artefato de deploy (FOWLER, 2023). Essa abordagem é especialmente adequada para projetos em estágio inicial, onde a sobrecarga de infraestrutura distribuída não se justifica, mas onde a organização modular facilita a evolução futura para microsserviços caso necessário.

Para sustentar essa operação, a arquitetura moderna baseia-se no desacoplamento entre cliente e servidor. Conforme documentado pela MDN Web Docs (2026), a separação entre um frontend SPA e um backend via API permite que cada parte do sistema evolua de forma independente, utilizando contratos de interface bem definidos. Essa abordagem não apenas favorece a escalabilidade, mas também garante que a lógica de negócio no servidor permaneça isolada da camada de apresentação do usuário.

A adoção de banco de dados híbrido, combinando sistemas relacionais e não relacionais, é motivada pela natureza heterogênea dos dados em plataformas de intermediação. Enquanto dados estruturados como usuários, anúncios e avaliações se beneficiam da integridade referencial de bancos relacionais, dados de mensagens de chat e logs de atividade têm características documentais que se alinham melhor ao modelo NoSQL (MONGODB, 2024).

2.4 Segurança e LGPD em Sistemas Web

A Lei Geral de Proteção de Dados Pessoais (LGPD — Lei nº 13.709/2018) estabelece o marco regulatório brasileiro para o tratamento de dados pessoais, impondo obrigações às organizações que coletam, processam ou armazenam informações de pessoas naturais. Em plataformas de intermediação entre usuários, dados como CPF, endereço e histórico de interações são classificados como dados pessoais e devem ser tratados com consentimento, finalidade definida e medidas técnicas de segurança (BRASIL, 2018).

A autenticação baseada em JSON Web Tokens (JWT) é amplamente adotada em sistemas RESTful por oferecer mecanismo stateless de verificação de identidade. O Spring Security fornece integração nativa com JWT e suporte a controle de acesso baseado em papéis (RBAC), atendendo aos requisitos de segurança exigidos pela LGPD (SPRING, 2024).

2.5 Tecnologias Utilizadas

2.5.1 PostgreSQL

O PostgreSQL é um sistema gerenciador de banco de dados relacional de código aberto com suporte avançado a consultas complexas, integridade referencial e transações ACID. Sua escolha para o armazenamento de dados estruturados do ReUseHub — como usuários, anúncios, categorias, avaliações e transações — justifica-se pela necessidade de garantir consistência e integridade dos relacionamentos entre essas entidades. O PostgreSQL integra-se nativamente ao ecossistema Spring por meio do Spring Data JPA e Hibernate (POSTGRESQL, 2024).

2.5.2 MongoDB

O MongoDB é um banco de dados orientado a documentos que armazena dados no formato BSON (Binary JSON), oferecendo flexibilidade de esquema e alto desempenho para dados não estruturados ou semi-estruturados. No ReUseHub, o MongoDB é utilizado para persistência de mensagens de chat, logs de atividade, histórico de interações e dados de recomendação, cujas características documentais e variabilidade de estrutura tornam inadequado o uso de esquemas relacionais rígidos (MONGODB, 2024).

2.5.3 Java 21 e Spring Boot

O Spring Boot é um framework do ecossistema Spring que simplifica a configuração e o desenvolvimento de aplicações Java, adotando o princípio de convenção sobre configuração. A adoção do Java 21 para o backend do ReUseHub deve-se à maturidade do ecossistema, à tipagem forte da linguagem, ao suporte nativo à arquitetura em camadas e à integração com Spring Security para autenticação e autorização. O Spring Data JPA abstrai o acesso ao PostgreSQL por meio de repositórios, enquanto a integração com o MongoDB é realizada via Spring Data MongoDB (SPRING, 2024).

2.5.4 React.js, TypeScript e Tailwind CSS

O React.js é uma biblioteca JavaScript para construção de interfaces de usuário baseadas em componentes, mantida pela Meta. A adoção do TypeScript adiciona tipagem estática ao desenvolvimento frontend, aumentando a segurança em tempo de compilação e facilitando a manutenção do código. O Tailwind CSS é um framework utilitário de CSS que permite a estilização diretamente nas classes HTML, acelerando o desenvolvimento da interface. Para gerenciamento de rotas da SPA, é adotado o React Router, e para visualização de dados no dashboard, o Chart.js (REACT, 2024; TAILWIND, 2024).

2.5.5 APIs Externas

O sistema integra APIs externas para complementar funcionalidades de endereço, geolocalização e comunicação por e-mail. A ViaCEP é utilizada para

consultar dados de endereço a partir do CEP informado, reduzindo a digitação manual de logradouro, bairro, cidade e estado (VIACEP, 2024). A Google Maps API apoia os recursos de geolocalização e visualização de mapas, permitindo trabalhar com informações geográficas relacionadas aos anúncios e usuários da plataforma (GOOGLE MAPS PLATFORM, 2026). Além disso, a Resend API é utilizada para envio de mensagens transacionais, especialmente no fluxo de recuperação e redefinição de senha dos usuários (RESEND, 2026).

3. MATERIAIS E MÉTODOS

3.1 Metodologia de Desenvolvimento

O desenvolvimento do ReUseHub seguiu uma abordagem iterativa e incremental, com ciclos curtos de entrega organizados em sprints semanais. A definição de escopo foi realizada previamente ao início da implementação, por meio de levantamento de requisitos funcionais e não funcionais, modelagem do banco de dados e definição da arquitetura do sistema.

O controle de versão foi realizado com Git, utilizando repositório hospedado no GitHub (<https://github.com/gustavoyoshizawaumc/ReUseHub>), permitindo rastreabilidade de alterações e colaboração entre os integrantes do projeto.

3.2 Arquitetura do Sistema

O sistema adota uma arquitetura de Monolito Modular, executada em um único processo e artefato de deploy, mas organizada internamente em módulos coesos e independentes, cada um responsável por um domínio de negócio específico. Essa abordagem combina a simplicidade operacional de um monolito — eliminando a complexidade de infraestrutura distribuída — com os princípios de separação de responsabilidades e baixo acoplamento característicos de arquiteturas orientadas a microsserviços (FOWLER, 2023).

Os módulos definidos no backend são: Autenticação, responsável pelo cadastro, autenticação JWT, recuperação de senha e controle de acesso por perfil; Anúncios, que gerencia o ciclo de vida dos itens publicados para doação ou troca; Interesses e Negociação, que controla solicitações de troca ou doação, aceite, entrega e confirmação de recebimento; Busca e Localização, que integra APIs externas e processa filtros geográficos; Chat, responsável pela troca de mensagens entre usuários com persistência no MongoDB; Avaliações, que gerencia reputação após a conclusão da negociação; Denúncias e Moderação, que tratam análise, aprovação, reprovação e suspensão de conteúdos; Administração, que concentra gestão de usuários, moderadores, métricas e auditoria; e Notificações, que comunica eventos relevantes aos participantes da plataforma.

O frontend é implementado como uma Single Page Application (SPA) em React.js com TypeScript e Tailwind CSS, desacoplado do backend e comunicando-se

exclusivamente via API RESTful. Essa separação garante independência entre as camadas de apresentação e de negócio, facilitando a evolução de cada uma sem impacto na outra.

3.3 Diagrama de Arquitetura

O diagrama de arquitetura de software é uma representação visual da estrutura de um sistema, destacando seus componentes, interações e dependências. Assim como uma planta baixa auxilia na construção de um edifício, o diagrama de arquitetura permite que desenvolvedores visualizem e compreendam a organização interna de um software (INFNET, 2025).

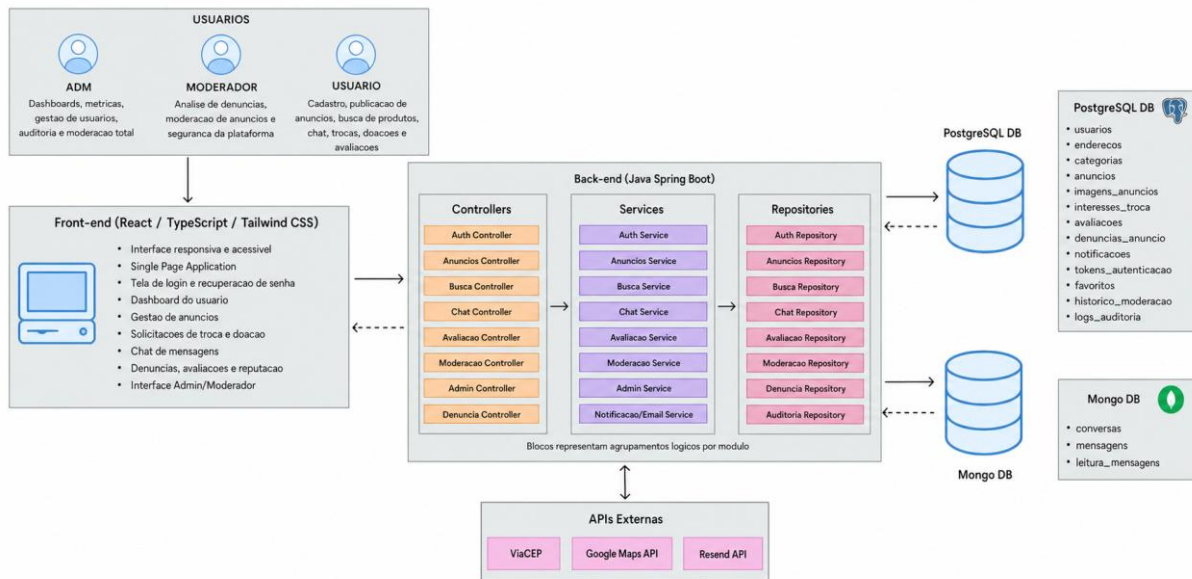
Conforme destacado pela Lucidchart (2021), os diagramas de arquitetura de software oferecem a toda a equipe de desenvolvimento um panorama visual, facilitando a comunicação de ideias e conceitos-chave em termos que todos entendem. Entre os principais benefícios estão a melhora na compreensão do sistema, o alinhamento estratégico entre equipes e partes interessadas, e a identificação antecipada de problemas e gargalos antes mesmos da implementação. Segundo a Microsoft (2024), arquitetos de software usam esses diagramas em diferentes etapas do ciclo de vida do projeto — do planejamento até a operação — sempre selecionando o tipo de diagrama que melhor corresponde à mensagem e ao público-alvo.

[INSERIR AQUI A IMAGEM DO DIAGRAMA DE ARQUITETURA]

Figura 1 – Diagrama de arquitetura do sistema ReUseHub.

Fonte: Autoria própria (2026).

No diagrama, os blocos de Controllers, Services e Repositories representam agrupamentos lógicos da aplicação. Cada bloco possui ramificações internas relacionadas aos principais módulos do sistema, como autenticação, anúncios, interesses, chat, avaliações, denúncias, moderação, administração e notificações. Essa representação foi adotada para manter a visualização macro da arquitetura sem poluir o diagrama com todos os componentes internos.



3.4 Modelagem de Dados

O modelo de dados do sistema é distribuído entre dois bancos de dados. No PostgreSQL, são armazenadas as entidades estruturadas e relacionais: usuarios, enderecos, categorias, anuncios, imagens_anuncios, interesses_troca, avaliacoes, denuncias_anuncio, notificacoes, tokens_autenticacao, favoritos, historico_moderacao e logs_auditoria. A entidade usuario é central para autenticação, controle de acesso e reputação, vinculando-se aos anúncios publicados, interesses registrados, avaliações recebidas e ações administrativas ou de moderação.

No MongoDB, são armazenados documentos relacionados à comunicação em tempo real, como conversas, mensagens e registros de leitura de mensagens. Essa divisão permite que dados transacionais e auditáveis permaneçam no PostgreSQL, enquanto o histórico de chat utiliza a flexibilidade documental do MongoDB, adequada para estruturas de mensagens e conversas (MONGODB, 2024; POSTGRES SQL, 2024).

1.2.3.5 Diagrama de Classes

O diagrama de classes representa a estrutura estática do sistema, demonstrando as principais classes, atributos, responsabilidades e relacionamentos entre entidades. Na UML, esse tipo de diagrama é utilizado para documentar a organização dos objetos de domínio e apoiar a comunicação entre análise, projeto e implementação (OBJECT MANAGEMENT GROUP, 2017).

No ReUseHub, o diagrama de classes deve contemplar entidades como Usuario, Anuncio, Categoria, Endereco, ImagemAnuncio, InteresseTroca, Avaliacao,

DenunciaAnuncio, Notificacao, Favorito, Conversa e Mensagem. Também devem ser indicados os relacionamentos centrais, como o vínculo entre usuário e anúncios publicados, anúncio e imagens, anúncio e interesses, usuários avaliadores e avaliados, além da associação entre conversas, mensagens e negociações.

[INSERIR AQUI A IMAGEM DO DIAGRAMA DE CLASSES]

Figura 2 – Diagrama de classes do sistema ReUseHub.

Fonte: Autoria própria (2026).

1.3.3.6 Diagrama de Atividades / BPMN

O BPMN (Business Process Model and Notation) é uma notação padronizada para representação de processos de negócio, permitindo demonstrar eventos, atividades, decisões, fluxos e participantes envolvidos em uma operação (OBJECT MANAGEMENT GROUP, 2014). No contexto do ReUseHub, esse diagrama é utilizado para documentar o fluxo de anúncio, interesse, negociação, entrega, confirmação de recebimento e avaliação entre usuários.

O fluxo principal inicia com o anúncio ativo, passa pelo registro de interesse por outro usuário, aceite do dono do anúncio, abertura automática do chat, negociação entre as partes, marcação de entrega pelo anunciante, confirmação de recebimento pelo interessado e liberação da avaliação. O anúncio somente é marcado como concluído após a confirmação das duas partes, reduzindo riscos de fraude e garantindo que a avaliação ocorra apenas após uma negociação efetivamente finalizada.

[INSERIR AQUI A IMAGEM DO BPMN DO FLUXO DE NEGOCIAÇÃO]

Figura 3 – BPMN do fluxo de anúncio, interesse, negociação, entrega e avaliação.

Fonte: Autoria própria (2026).

3.7 Funcionalidades Principais

O sistema é composto por seis funcionalidades principais, distribuídas entre os dois integrantes do projeto. O cadastro e autenticação de usuários utiliza Spring Security com JWT para emissão e validação de tokens, implementando controle de acesso baseado em perfis. A publicação e gerenciamento de anúncios permite ao

usuário criar, editar, pausar e encerrar anúncios de itens disponíveis para doação ou troca.

A busca com filtros por categoria e localização utiliza as APIs ViaCEP e Google Maps API para enriquecer os dados de endereço e permitir filtragem geográfica dos anúncios. Além disso, o sistema utiliza a Resend API no fluxo de recuperação de senha, enviando mensagens transacionais aos usuários cadastrados.

O chat entre usuários é implementado com persistência das mensagens no MongoDB, permitindo comunicação direta entre o publicador de um anúncio e os usuários interessados. O sistema de avaliações e reputação permite que, após a conclusão de uma transação, ambos os usuários envolvidos avaliem a experiência, construindo um histórico de reputação vinculado ao perfil de cada usuário.

3.8 Segurança e Conformidade com a LGPD

A autenticação é implementada com JWT, emitido pelo backend após validação das credenciais e enviado pelo frontend em todas as requisições subsequentes no cabeçalho Authorization como Bearer token. O Spring Security intercepta cada requisição antes de chegar aos controllers, validando o token e extraíndo o perfil do usuário para aplicação das regras de autorização por papel (RBAC).

Em conformidade com a LGPD, dados sensíveis como senhas são armazenados exclusivamente em formato hash via BCrypt. Dados pessoais como CPF e endereço são tratados com finalidade explícita de identificação e geolocalização cadastral. O sistema implementa os princípios de minimização de dados e finalidade no tratamento das informações dos usuários.

4. EAP

A Estrutura Analítica do Projeto (EAP), conhecida em inglês como Work Breakdown Structure (WBS), é uma ferramenta de gestão de projetos usada para definir e gerenciar as entregas de um projeto. Trata-se de uma estrutura hierárquica que divide atividades complexas em partes mais gerenciáveis, mostrando cada entrega que precisa ser concluída para atingir a meta geral do projeto (LUCIDCHART, 2024). Enquanto a maioria das ferramentas de gestão de projetos foca em ações planejadas, a EAP trabalha com resultados planejados, o que possibilita estimativas mais precisas de custos, prazos e recursos (ARTIA, 2025).

Conforme definido pelo Project Management Institute (PMI, 2021) no Guia PMBOK®, a EAP é estruturada em árvore exaustiva e hierárquica — do mais geral para o mais específico — orientada às entregas ou fases do ciclo de vida que precisam

ser concluídas para completar o projeto. Um dos aspectos mais importantes da EAP é a Regra dos 100%, que estabelece que ela deve incluir 100% do trabalho definido pelo escopo do projeto, capturando todas as entregas internas, externas e intermediárias (ESCRITORIO DE PROJETOS, 2024). Além disso, cada elemento da estrutura possui um código único que identifica seu nível e sua relação hierárquica com os demais elementos.

No contexto do ReUseHub, a EAP organiza as funcionalidades e tarefas do projeto em dois grandes grupos: as funcionalidades do sistema — como autenticação, gerenciamento de anúncios, busca, chat e avaliações — e as tarefas transversais obrigatórias, como segurança e conformidade com a LGPD, testes automatizados e elaboração do manual do sistema. Essa estrutura facilita a distribuição de responsabilidades entre os integrantes da equipe e o acompanhamento do progresso de cada entrega ao longo do desenvolvimento.

1.0 ReUseHub

Funcionalidades do Sistema

- **1.1 Login** *(Obrigatório)*
- **1.2 Sistema de Cadastro e Autenticação de Usuários** — *Gustavo*
- **1.3 Gerenciador de Anúncios (Doação/Troca)** — *Guilherme*
- **1.4 Motor de Busca e Filtros** — *Guilherme*
- **1.5 Sistema de Destaque de Anúncios** — *Guilherme*
- **1.6 Ferramenta de Chat** — *Gustavo*
- **1.7 Sistema de Avaliações e Reputação** — *Gustavo*
- **1.8 Dashboard Analítico** *(Obrigatório)*

Tarefas do Projeto

- **1.9 Segurança e Conformidade com a LGPD** *(Obrigatório)*
- **1.10 Testes Automatizados** *(Obrigatório)*
- **1.11 Manual do Sistema** *(Obrigatório)*

1.1 Tela de Login e Acesso

- **1.1.1** Projetar a interface da página de login com React.js e Tailwind CSS.
- **1.1.2** Codificar formulário de e-mail e senha com validação.
- **1.1.3** Adicionar tratamento visual de erros de autenticação.
- **1.1.4** Integrar a tela ao endpoint de login da API.
- **1.1.5** Gerenciar o token JWT no frontend.
- **1.1.6** Implementar logout no lado do cliente.

1.2 Cadastro, Autenticação e Perfil de Usuários — Gustavo

- **1.2.1** Criar a entidade usuario no PostgreSQL com dados pessoais, perfil e consentimento LGPD.
- **1.2.2** Implementar cadastro com validação de dados obrigatórios.
- **1.2.3** Implementar hash de senha com BCrypt.
- **1.2.4** Configurar Spring Security com emissão e validação de JWT.
- **1.2.5** Definir controle de acesso por perfil (Usuário, Moderador e Administrador).
- **1.2.6** Implementar edição de perfil, incluindo telefone, biografia e foto.
- **1.2.7** Implementar alteração de senha.
- **1.2.8** Implementar exclusão/desativação da conta do usuário.

1.3 Gerenciador de Anúncios — Guilherme

- **1.3.1** Desenvolver formulário de publicação de anúncio.
- **1.3.2** Permitir anúncios de doação ou troca.
- **1.3.3** Relacionar anúncio com usuário, categoria e endereço.
- **1.3.4** Implementar upload, armazenamento e otimização de imagens.
- **1.3.5** Permitir edição, pausa, encerramento e exclusão dos próprios anúncios.
- **1.3.6** Criar página de detalhes do anúncio.
- **1.3.7** Submeter anúncios à aprovação da moderação antes da publicação.

1.4 Busca, Filtros e Localização — Guilherme

- **1.4.1** Desenvolver busca por palavras-chave com Full Text Search no PostgreSQL.
- **1.4.2** Implementar filtros por categoria, tipo e condição do item.
- **1.4.3** Permitir busca por CEP informado na Home.
- **1.4.4** Integrar ViaCEP para validação e preenchimento de endereço.
- **1.4.5** Integrar Google Maps API para obtenção de coordenadas geográficas.
- **1.4.6** Implementar filtro de anúncios por raio de distância.
- **1.4.7** Implementar paginação e ordenação dos resultados.

1.5 Destaque e Relevância de Anúncios — Guilherme

- **1.5.1** Definir critérios de relevância dos anúncios.
- **1.5.2** Calcular score com base em visualizações, recência, interesses e atividade.
- **1.5.3** Ordenar os melhores anúncios primeiro na listagem.
- **1.5.4** Exibir marcações visuais para anúncios destacados, quando aplicável.

1.6 Interesses e Negociação — Guilherme/Gustavo

- **1.6.1** Permitir manifestação de interesse em anúncios ativos.
- **1.6.2** Permitir envio de mensagem junto ao interesse.
- **1.6.3** Permitir oferta de outro anúncio em propostas de troca.
- **1.6.4** Permitir que o publicador aceite ou recuse interesses recebidos.
- **1.6.5** Criar ou recuperar conversa após aceite do interesse.
- **1.6.6** Permitir marcação de item como entregue.

- **1.6.7** Permitir confirmação de recebimento.
- **1.6.8** Concluir a negociação após confirmação dos envolvidos.

1.7 Ferramenta de Chat — Gustavo

- **1.7.1** Modelar documentos de conversa e mensagens no MongoDB.
- **1.7.2** Desenvolver interface de conversas privadas no frontend.
- **1.7.3** Implementar envio e recuperação de mensagens.
- **1.7.4** Listar conversas do usuário autenticado.
- **1.7.5** Marcar mensagens/conversas como lidas.
- **1.7.6** Bloquear novas mensagens após o fechamento da negociação, quando aplicável.

1.8 Avaliações e Reputação — Gustavo

- **1.8.1** Criar formulário de avaliação com nota de 1 a 5 estrelas.
- **1.8.2** Permitir envio de comentário textual.
- **1.8.3** Liberar avaliação apenas após conclusão da negociação.
- **1.8.4** Impedir avaliações duplicadas.
- **1.8.5** Calcular nota média consolidada do usuário.
- **1.8.6** Exibir reputação e histórico de avaliações no perfil público.

1.9 Denúncias e Moderação

- **1.9.1** Permitir denúncia de anúncios inadequados.
- **1.9.2** Listar denúncias para Moderador e Administrador.
- **1.9.3** Permitir análise e descarte de denúncias com justificativa.
- **1.9.4** Permitir suspensão, reativação ou reprovação de anúncios denunciados.
- **1.9.5** Permitir aprovação ou reprovação de anúncios pendentes.
- **1.9.6** Registrar histórico das ações de moderação.

- **1.9.7** Permitir remoção de avaliações inadequadas.

1.10 Notificações

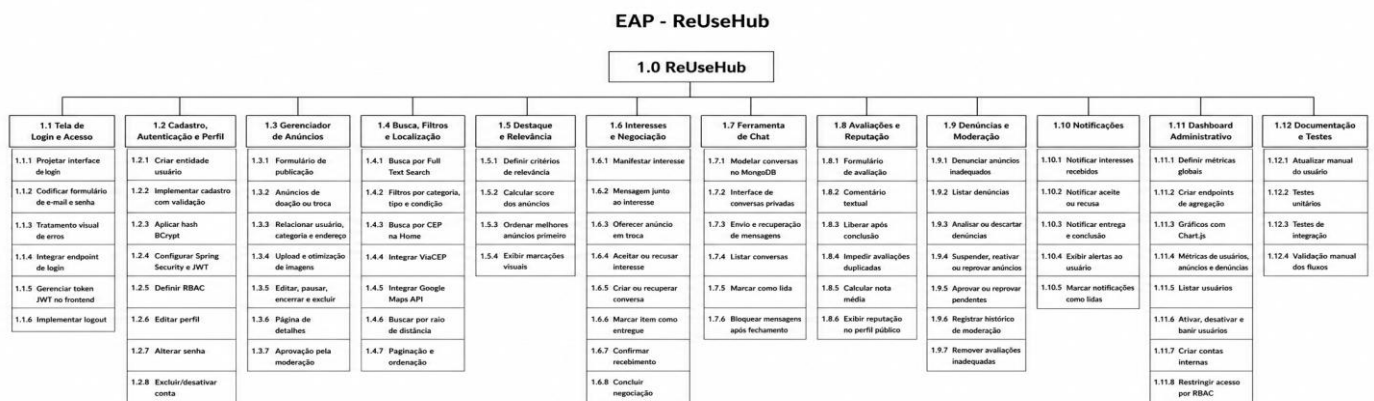
- **1.10.1** Criar notificações para interesses recebidos.
- **1.10.2** Criar notificações para interesses aceitos ou recusados.
- **1.10.3** Criar notificações para entrega marcada e negociação concluída.
- **1.10.4** Exibir notificações ou alertas de eventos relevantes ao usuário.
- **1.10.5** Marcar notificações como lidas, se implementado.

1.11 Dashboard Administrativo

- **1.11.1** Definir métricas globais do sistema.
- **1.11.2** Criar endpoints de agregação no backend.
- **1.11.3** Implementar gráficos com Chart.js.
- **1.11.4** Exibir métricas de usuários, anúncios, denúncias e negociações concluídas.
- **1.11.5** Listar usuários para administração.
- **1.11.6** Permitir ativação, desativação e banimento de usuários.
- **1.11.7** Permitir criação de contas de Moderador e Administrador.
- **1.11.8** Restringir acesso ao painel por RBAC.

1.12 Documentação e Testes

- **1.12.1** Criar ou atualizar manual do usuário.
- **1.12.2** Criar testes unitários para regras de negócio.
- **1.12.3** Criar testes de integração para endpoints principais.
- **1.12.4** Validar fluxos principais do sistema de forma manual.



2 Requisitos do Sistema.

Os requisitos de um sistema constituem as descrições fundamentais do que o software deve realizar, abrangendo os serviços oferecidos e as restrições impostas ao seu funcionamento. Enquanto os requisitos funcionais definem o que o sistema deve fazer para atender às necessidades do usuário, os requisitos não funcionais definem as propriedades emergentes, como desempenho, segurança e confiabilidade, além das restrições de desenvolvimento que asseguram a qualidade do produto final (SOMMERVILLE, 2018).

2.1 – Requisitos Funcionais

Requisitos Funcionais	Funcionalidade
RF01	Cadastro de novos usuários com nome, e-mail, CPF, telefone, senha e aceite da LGPD.

RF02	Autenticação de usuários por e-mail e senha.
RF03	Recuperação de senha via envio de link/token por e-mail.
RF04	Geração e validação de token JWT para acesso às funcionalidades protegidas
RF05	Controle de acesso por perfis: usuário comum, moderador e administrador.
RF06	Visualização dos dados da própria conta pelo usuário.
RF07	Edição do perfil do usuário (nome, telefone, biografia e avatar).
RF08	Anonimização da própria conta a pedido do usuário, preservando o histórico de terceiros (direito do titular — LGPD, art. 18).
RF09	Perfil público de usuário com reputação, avaliações e anúncios vinculados.
RF10	Publicação de anúncios de doação ou troca por usuários autenticados.
RF11	Cadastro de título, descrição, categoria, condição, tipo, endereço e data de expiração do anúncio.
RF12	Upload e armazenamento de imagens relacionadas aos anúncios.
RF13	Listagem pública de anúncios ativos.
RF14	Visualização detalhada de um anúncio.
RF15	Visualização dos próprios anúncios pelo usuário autenticado.
RF16	Edição dos próprios anúncios pelo proprietário.
RF17	Alteração do status permitido dos próprios anúncios pelo proprietário.
RF18	Exclusão dos próprios anúncios pelo proprietário.
RF19	Submissão de anúncios recém-criados à aprovação da moderação antes da publicação.
RF20	Aprovação ou reprovação de anúncios pendentes por moderadores ou administradores.

RF21	Busca de anúncios por palavra-chave.
RF22	Filtro de anúncios por categoria.
RF23	Filtro de anúncios por tipo: doação ou troca.
RF24	Filtro de anúncios por condição do item.
RF25	Busca e filtragem de anúncios por localização geográfica (CEP convertido em coordenadas e raio).
RF26	Integração com ViaCEP (validação/enriquecimento de endereço) e OpenStreetMap Nominatim (geocodificação).
RF27	Ordenação de anúncios por relevância, distância, recentes e populares.
RF28	Cálculo automático da relevância de anúncios (recência, interesses, visualizações e atividade do publicador).
RF29	Destaque de anúncios relevantes na listagem principal.
RF30	Selos visuais para anúncios destacados ("Novo", "Em alta" ou "Muito procurado").
RF31	Favoritar anúncios e visualizar a lista de favoritos do usuário autenticado.
RF32	Manifestação de interesse em um anúncio ativo.
RF33	Envio de mensagem junto à manifestação de interesse.
RF34	Oferta de outro anúncio próprio em uma proposta de troca.
RF35	Impedimento de manifestação de interesse no próprio anúncio.
RF36	Visualização dos interesses recebidos pelo dono do anúncio.
RF37	Visualização dos interesses enviados pelo usuário.
RF38	Aceite ou recusa de uma manifestação de interesse pelo dono do anúncio.
RF39	Início ou recuperação de uma conversa quando um interesse for aceito.

RF40	Marcação do item como entregue pelo dono.
RF41	Confirmação do recebimento do item pelo interessado.
RF42	Conclusão da negociação após confirmação de recebimento.
RF43	Início de conversa entre usuários sobre um anúncio ativo.
RF44	Envio de mensagens privadas entre usuários autenticados.
RF45	Persistência do histórico de conversas no MongoDB.
RF46	Visualização das conversas do usuário.
RF47	Abertura de uma conversa específica e visualização de seu histórico.
RF48	Marcação de conversas como lidas.
RF49	Indicação de mensagens ou conversas não lidas.
RF50	Bloqueio do envio de novas mensagens após o fechamento da negociação, quando aplicável.
RF51	Avaliação de usuários após uma negociação concluída.
RF52	Avaliação com nota de 1 a 5 estrelas e comentário textual.
RF53	Impedimento de avaliações duplicadas na mesma negociação.
RF54	Restrição da avaliação apenas aos participantes da negociação.
RF55	Cálculo da reputação média do usuário avaliado.
RF56	Exibição das avaliações recebidas no perfil público do usuário.
RF57	Denúncia de anúncios por usuários autenticados.
RF58	Impedimento de denúncia do próprio anúncio.

RF59	Visualização de denúncias por status por moderadores e administradores.
RF60	Análise ou descarte de denúncias por moderadores e administradores.
RF61	Registro de justificativa nas ações de moderação.
RF62	Listagem de anúncios suspeitos com base na quantidade de denúncias abertas.
RF63	Suspensão, reativação ou reprovação definitiva de anúncios por moderadores e administradores.
RF64	Remoção de avaliações inadequadas por moderadores e administradores.
RF65	Registro do histórico das ações de moderação.
RF66	Visualização do próprio histórico de ações pelo moderador
RF67	Criação de contas internas de moderador pelo administrador.
RF68	Criação de contas internas de administrador pelo administrador.
RF69	Listagem de usuários cadastrados pelo administrador.
RF70	Filtragem de usuários por termo, perfil e status pelo administrador.
RF71	Ativação, desativação ou banimento de usuários pelo administrador (controle de acesso, reversível).
RF72	Anonimização de conta de usuário pelo administrador, preservando o histórico de terceiros (LGPD).
RF73	Impedimento de o administrador desativar, banir ou excluir a própria conta administrativa.
RF74	Dashboard administrativo com métricas globais (usuários, anúncios, denúncias e negociações concluídas) e gráficos via Chart.js.
RF75	Ranking de anúncios mais visualizados.
RF76	Ranking de usuários com melhor reputação.
RF77	Consulta de logs de auditoria pelo administrador.

RF78	Geração ou filtragem de relatórios por período selecionado.
RF79	Visualização de estatísticas dos próprios anúncios pelo usuário.
RF80	Criação de notificações para eventos relevantes (interesse recebido, aceito, recusado e negociação concluída).
RF81	Visualização das notificações pelo usuário.
RF82	Marcação de notificações como lidas.
RF83	Exibição de alertas de mensagens não lidas.
RF84	Disponibilização de política ou aviso de privacidade relacionado à LGPD.
RF85	Registro do consentimento LGPD no cadastro.
RF86	Disponibilização de manual do sistema para usuários.

Fonte: Autor próprio (2026).

2.2 Requisitos Não Funcionais.

Requisitos Não Funcionais	Funcionalidade
RNF01	O sistema deve implementar autenticação via JWT com criptografia de senhas utilizando o algoritmo BCrypt.
RNF02	O sistema deve garantir a proteção nativa contra ataques CSRF (Cross-Site Request Forgery) através do Spring Security.
RNF03	O sistema deve estar em conformidade com a Lei Geral de Proteção de Dados (LGPD) no tratamento e armazenamento de informações dos usuários.
RNF04	O sistema deve garantir que consultas ao PostgreSQL sejam otimizadas para manter o tempo de resposta abaixo de 300 milissegundos.
RNF05	O sistema deve utilizar a estratégia de persistência híbrida (PostgreSQL para dados relacionais e MongoDB para histórico de mensagens) para otimizar o acesso aos dados.
RNF06	O sistema deve manter uma arquitetura de Monolito Modular para permitir a evolução e possível extração de módulos em microserviços.

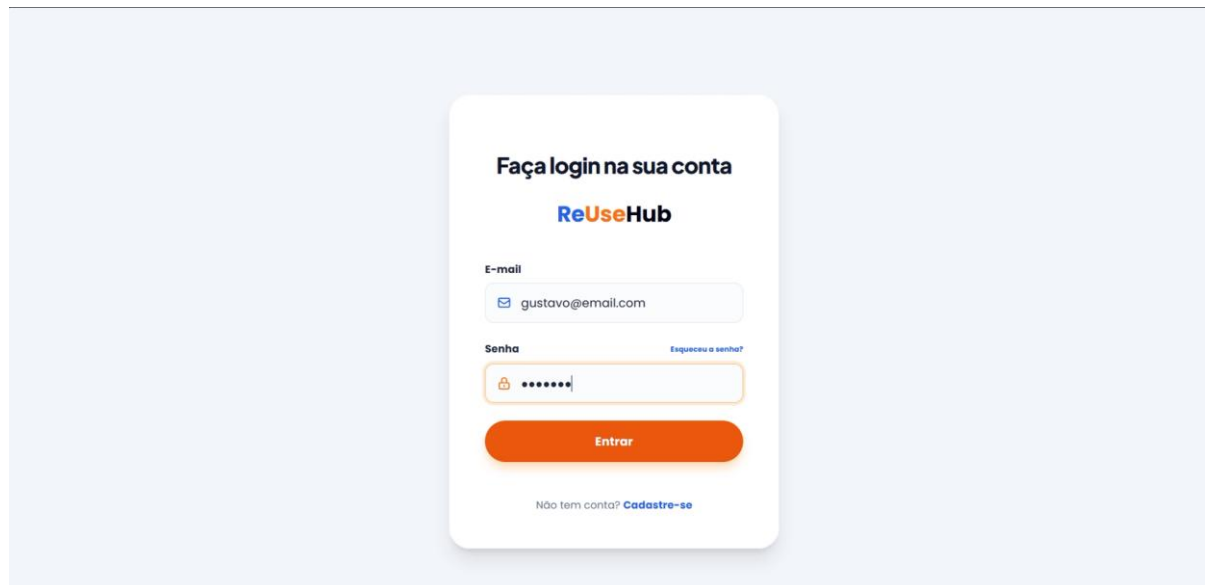
RNF07	O sistema deve expor suas funcionalidades através de uma API RESTful, garantindo o desacoplamento total entre o frontend (React) e o backend (Spring Boot).
RNF08	O sistema deve possuir uma interface responsiva, adaptando-se a dispositivos móveis (390px) e desktops (>768px).
RNF09	O sistema deve utilizar o framework Tailwind CSS para garantir a padronização visual e a eficiência no desenvolvimento da interface.
RNF10	O sistema deve garantir a integridade referencial dos dados através das restrições do banco de dados PostgreSQL.
RNF11	O sistema deve possuir cobertura de testes automatizados (unitários e de integração) utilizando JUnit e Mockito para garantir a estabilidade das regras de negócio.

Fonte: Autor próprio (2026).

2.3 Protótipos

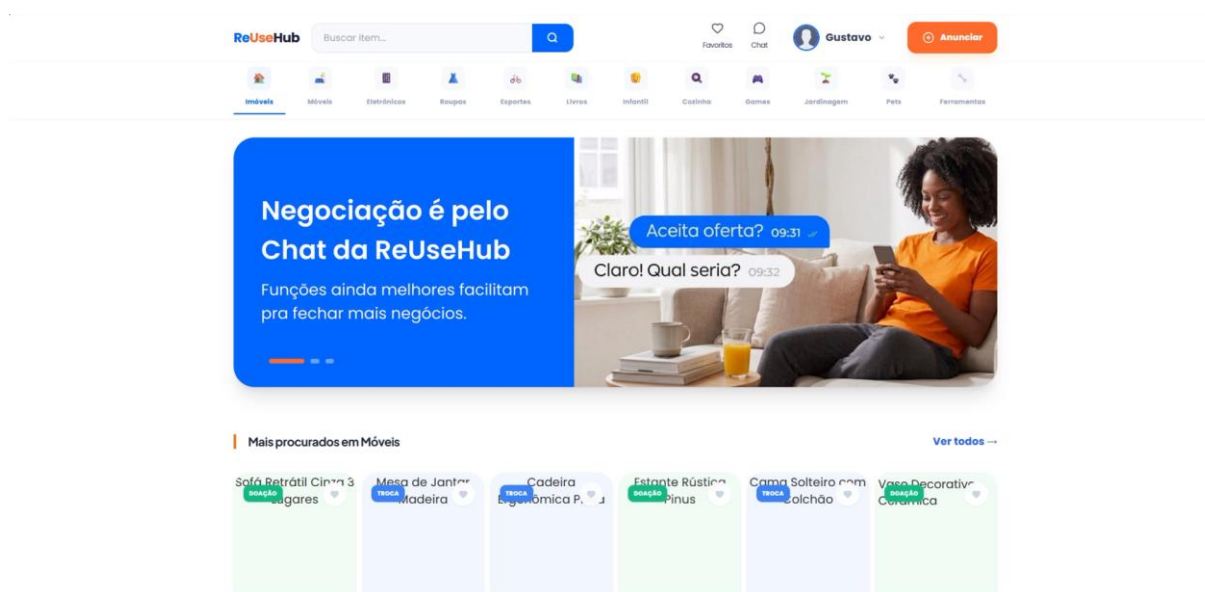
Os protótipos apresentados a seguir detalham o login do sistema e o fluxo de navegabilidade do usuário. As capturas de tela exibem a aplicação em ambiente de desenvolvimento, cuja interface foi construída através da biblioteca React.js, utilizando a abordagem *utility-first* do framework Tailwind CSS para garantir um design responsivo e consistente.

Figura 4 – Tela de Login: interface inicial.



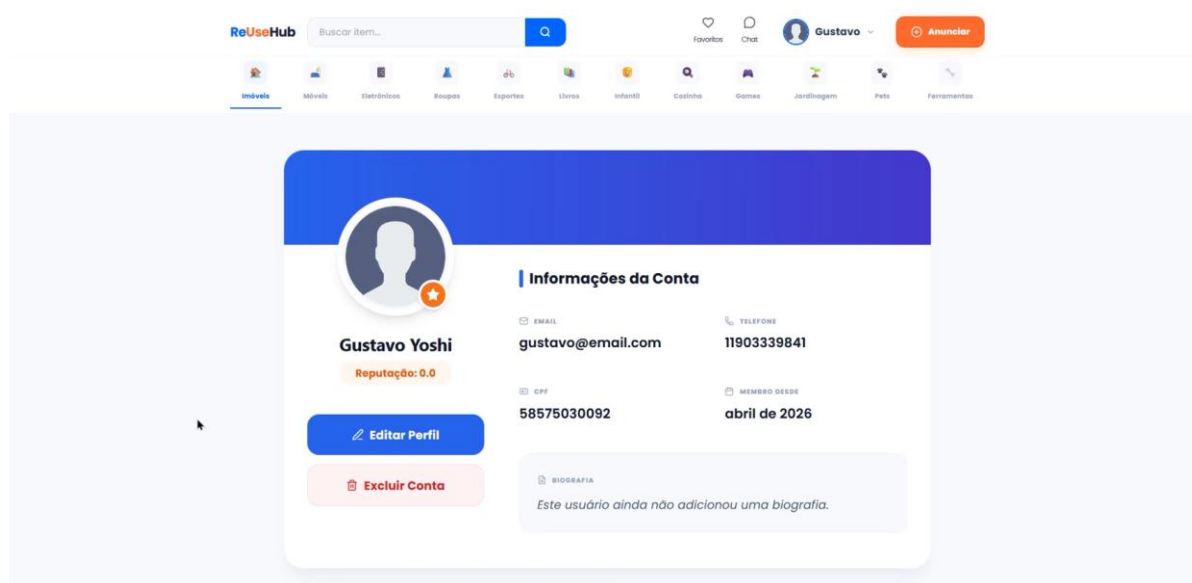
Fonte: Autoria própria (2026).

Figura 2 - Tela Inicial (Home).



Fonte: Autoria própria (2026).

Figura 5 – Tela de Perfil do Usuário.



Fonte: Autoria própria (2026).

5. FUNCIONALIDADES DESENVOLVIDAS

Cadastro e Autenticação de Usuários

O módulo de cadastro e autenticação foi implementado utilizando Spring Security, um framework robusto para autenticação e autorização em aplicações Java que oferece proteção contra diversos tipos de ataques comuns em aplicações web. De acordo com SPRING SECURITY FRAMEWORK (2024), o Spring Security fornece uma arquitetura em camadas que permite autenticação através de múltiplos mecanismos, incluindo formulários, tokens e protocolos OAuth. No contexto do ReUseHub, o sistema implementa um fluxo de registro onde novos usuários fornecem email, senha, CPF e telefone. Estes dados são validados no backend através de verificações de email único e CPF válido, garantindo integridade dos dados armazenados no banco de dados PostgreSQL. O Spring Security gerencia toda a lógica de autenticação, desde a captura de credenciais até a injeção automática do contexto de segurança em cada requisição autenticada.

A escolha de Spring Security alinha-se com as melhores práticas de segurança recomendadas por OWASP (2024), que destaca a importância de autenticação robusta como primeira linha de defesa contra acessos não autorizados. O framework

fornece proteção automática contra vulnerabilidades comuns como CSRF (Cross-Site Request Forgery) e implementa validação de entrada para prevenir injection attacks. Durante o processo de registro, as informações do usuário são armazenadas de forma segura, com a senha sendo processada através de um algoritmo criptográfico adaptativo antes de ser persistida no banco de dados. Este processo garante conformidade com a Lei Geral de Proteção de Dados (LGPD), exigindo consentimento explícito do usuário para armazenamento de dados pessoais e oferecendo controle sobre informações armazenadas.

Publicação e Gerenciamento de Anúncios (Doação/Troca)

O módulo de anúncios implementa funcionalidades completas de CRUD (Create, Read, Update, Delete), permitindo que usuários autenticados publiquem itens para doação ou troca. Conforme HIBERNATE FRAMEWORK (2024), o framework utilizado para mapeamento objeto-relacional (ORM), o sistema armazena anúncios no banco de dados PostgreSQL com relacionamentos definidos entre entidades (Anúncio, Usuário, Categoria, Endereço). Cada anúncio contém informações estruturadas incluindo título, descrição, tipo (Doação ou Troca), condição do item (Novo, Bom, Regular, Ruim), categoria, localização (integrada com API ViaCEP para validação automática de endereços) e data de expiração. O sistema implementa validações rigorosas tanto no frontend quanto no backend, garantindo que apenas dados de qualidade sejam persistidos. A listagem de anúncios oferece múltiplos endpoints para descoberta: listagem geral ordenada por relevância, busca por termo-chave, filtros por categoria e tipo, e visualização de detalhes completos.

A funcionalidade de gerenciamento garante que apenas o proprietário de um anúncio possa editá-lo ou deletá-lo, através de verificação de propriedade realizada no nível de serviço. De acordo com POSTGRESQL DOCUMENTATION (2024), o banco de dados PostgreSQL oferece integridade referencial e constraints que garantem consistência dos relacionamentos entre tabelas. A integração com ViaCEP, conforme documentado por VIACEP (2024), automatiza o processo de validação e preenchimento de endereços, eliminando erros manuais e melhorando a qualidade dos dados geográficos. O sistema suporta paginação e ordenação configurável em

todas as listagens, permitindo escalabilidade mesmo com volume elevado de anúncios. Adicionalmente, implementa controle de acesso granular, onde operações de leitura são públicas (permitindo descoberta antes do registro) enquanto operações de criação e modificação exigem autenticação e verificação de propriedade.

Login com Segurança e Criptografia

O sistema de login implementa autenticação stateless utilizando JWT (JSON Web Tokens), um padrão internacional definido por RFC 7519 (2015) que estabelece um formato compacto e auto-contido para transmissão segura de informações entre partes. Conforme especificado pela IETF (2015), um JWT consiste em três partes codificadas em Base64URL separadas por pontos: header (tipo de token e algoritmo), payload (claims contendo dados do usuário) e signature (garantia de integridade). No ReUseHub, quando um usuário realiza login fornecendo email e senha, o backend valida as credenciais e, em caso de sucesso, gera um token JWT contendo o email do usuário com tempo de expiração configurável (padrão 24 horas). Este token é retornado ao cliente, armazenado localmente e incluído no header Authorization de todas as requisições subsequentes. O servidor valida o token em cada requisição através de um filtro customizado que verifica a assinatura criptográfica e validade temporal, rejeitando requisições com tokens inválidos ou expirados com status 401.

A criptografia de senhas é implementada através do algoritmo BCrypt, um algoritmo de hash adaptativo recomendado por OWASP (2024) e documentado em RFC 2898 (2000) como função de derivação de chave. De acordo com BCrypt OFFICIAL DOCUMENTATION (2023), o BCrypt aplica automaticamente múltiplas rodadas de hash (padrão 10 rodadas) e incorpora um "salt" aleatório único a cada senha, tornando computacionalmente impraticável ataques de força bruta ou utilização de tabelas de hash pré-calculadas (rainbow tables). Durante o registro, a senha fornecida é processada através do BCrypt antes de ser armazenada, enquanto na autenticação a senha fornecida é comparada com o hash armazenado sem necessidade de reversão. A implementação de JWT stateless oferece vantagens significativas em arquiteturas distribuídas: não requer sincronização de estado entre servidores, permite escalabilidade horizontal e oferece proteção inerente contra CSRF, pois tokens devem

ser incluídos explicitamente em headers HTTP (não enviados automaticamente pelo navegador em requisições cross-origin).

6. REFERÊNCIAS BIBLIOGRÁFICAS

REACT. React documentation. Meta Open Source, 2024. Disponível em: <https://react.dev>. Acesso em: mar. 2026.

BRITANNICA, The Editors of Encyclopaedia. Information system. Encyclopedia Britannica, 2026. Disponível em: <https://www.britannica.com/topic/information-system>. Acesso em: mar. 2026.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Diário Oficial da União, Brasília, DF, 15 ago. 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: mar. 2026.

MONGODB. Introduction to MongoDB. MongoDB, Inc., 2024. Disponível em: <https://www.mongodb.com/docs/manual/introduction/>. Acesso em: mar. 2026.

ELLEN MACARTHUR FOUNDATION. Circular economy introduction. 2023. Disponível em: <https://ellenmacarthurfoundation.org>. Acesso em: mar. 2026.

MDN WEB DOCS. Identificação de recursos e separação de preocupações em arquiteturas Web. Mozilla Developer Network, 2026. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Overview>. Acesso em: mar. 2026.

FOWLER, Martin. Monolith first. martinowler.com, 2015. Disponível em: <https://martinfowler.com/bliki/MonolithFirst.html>. Acesso em: mar. 2026.

OECD. An introduction to online platforms and their role in the digital transformation. Paris: OECD Publishing, 2019. Disponível em: <https://doi.org/10.1787/53e5f593-en>. Acesso em: mar. 2026.

POSTGRESQL. PostgreSQL 16 Documentation. The PostgreSQL Global Development Group, 2024. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: mar. 2026.

TAILWIND CSS. Tailwind CSS documentation. 2024. Disponível em: <https://tailwindcss.com/docs>. Acesso em: mar. 2026.

VIACEP. ViaCEP: webservice gratuito de CEP do Brasil. 2024. Disponível em: <https://viacep.com.br>. Acesso em: mar. 2026.

SPRING. Spring Boot reference documentation. VMware, Inc., 2024. Disponível em: <https://docs.spring.io/spring-boot/index.html>. Acesso em: mar. 2026.

ARTIA. EAP: o que é e como fazer uma Estrutura Analítica do Projeto em 4 passos. Artia, 2025. Disponível em: <https://artia.com/blog/como-fazer-eap-na-gestao-de-projetos/>. Acesso em: mar. 2026

ESCRITORIO DE PROJETOS. EAP — Estrutura Analítica do Projeto. Escritório de Projetos, 2024. Disponível em: <https://escritoriodeprojetos.com.br/eap/>. Acesso em: mar. 2026

INFNET. O que é e para que serve o diagrama de Arquitetura de Software. Blog Infnet, 2025. Disponível em: https://blog.infnet.com.br/arquitetura_software/o-que-e-e-para-que-serve-o-diagrama-de-arquitetura-de-software/. Acesso em: mar. 2026.

LUCIDCHART. 5 diagramas de arquitetura de sistema. Lucidchart Blog, 2021. Disponível em: <https://www.lucidchart.com/blog/pt/como-fazer-diagramas-de-arquitetura-de-sistema>. Acesso em: mar. 2026.

LUCIDCHART. O que é e como fazer uma EAP (WBS). Lucidchart, 2024. Disponível em: <https://www.lucidchart.com/pages/pt/eap-estrutura-analitica-do-projeto>. Acesso em: mar. 2026.

MICROSOFT. Criar diagramas de design de arquitetura. Microsoft Azure Well-Architected Framework, 2024. Disponível em: <https://learn.microsoft.com/pt-br/azure/well-architected/architect-role/design-diagrams>. Acesso em: mar. 2026.

PROJECT MANAGEMENT INSTITUTE. Um guia do conhecimento em gerenciamento de projetos (Guia PMBOK®). 7. ed. Newtown Square: Project Management Institute, 2021.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2018.

BCRYPT OFFICIAL DOCUMENTATION. Bcrypt Password Hashing. 2023. Disponível em: <https://bcrypt.online/>. Acesso em: abr. 2026.

HIBERNATE FRAMEWORK. Hibernate Object/Relational Mapping. Red Hat, 2024. Disponível em: <https://hibernate.org/>. Acesso em: abr. 2026.

IETF. RFC 2898 - PKCS #5: Password-Based Cryptography Specification. Internet Engineering Task Force, 2000. Disponível em: <https://tools.ietf.org/html/rfc2898>. Acesso em: abr. 2026.

IETF. RFC 7519 - JSON Web Token (JWT). Internet Engineering Task Force, 2015. Disponível em: <https://tools.ietf.org/html/rfc7519>. Acesso em: abr. 2026.

OWASP. OWASP Top 10 Web Application Security Risks. Open Web Application Security Project, 2024. Disponível em: <https://owasp.org/Top10/>. Acesso em: abr. 2026.

POSTGRESQL DOCUMENTATION. PostgreSQL Official Documentation. 2024. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: abr. 2026.

SPRING SECURITY FRAMEWORK. Spring Security Architecture and Reference. Spring.io, 2024. Disponível em: <https://spring.io/projects/spring-security>. Acesso em: abr. 2026.

VIACEP. ViaCEP - Webservice de CEP. 2024. Disponível em: <https://viacep.com.br/>. Acesso em: abr. 2026.

GOOGLE MAPS PLATFORM. Google Maps Platform documentation. Google, 2026. Disponível em: <https://developers.google.com/maps/documentation>. Acesso em: maio 2026.

RESEND. Resend documentation. Resend, 2026. Disponível em: <https://resend.com/docs>. Acesso em: maio 2026.

OBJECT MANAGEMENT GROUP. OMG Unified Modeling Language (OMG UML), version 2.5.1. Object Management Group, 2017. Disponível em: <https://www.omg.org/spec/UML/2.5.1/>. Acesso em: maio 2026.

OBJECT MANAGEMENT GROUP. Business Process Model and Notation (BPMN), version 2.0.2. Object Management Group, 2014. Disponível em: <https://www.omg.org/spec/BPMN/2.0.2/>. Acesso em: maio 2026.